Sign Language to Speech Handglove

Physics Project – Grade 12



Prepared by:

Rahul Trivedi

Yug Bhimani

Darsh Ramani

Class: 12-A

Subject: Physics

Acknowledgement

We would like to express our deepest gratitude to our respected Physics teacher Priti madam and Lab instructors Sanjay sir and Nilesh Sir, for their invaluable guidance, constant supervision, and constructive suggestions throughout the development of this investigative project. Their immense knowledge and logical insights helped us navigate the complex hardware integrations and coding pipelines.

We are highly indebted to our parents and peers for their continuous support, encouragement, and assistance in gathering the components and compiling this detailed report.

Lastly, We would like to thank the Council for the Indian School Certificate Examinations (CISCE) for incorporating investigative projects into the Class 12 Physics curriculum, which has allowed us to apply theoretical physics concepts to solve real-world, human-centric challenges.

Index

Section	Pg No.
Preface	2
Introduction of the Project	3 - 4
History and Background of the Topic	5 - 6
Component Overview and Hardware Connections	7 - 11
Theory and Principles Involved	12 - 18
Analysis, Observations, Calculations and Graph Work	19 - 30
System Code and Architecture	31 - 36
Evidence of Own Work	37
Conclusion	38
Scope of Improvement	39
Bibliography	40

Preface

In the modern era of rapid technological advancement, the true success of scientific research lies in its capacity to serve humanity. This physics project, titled **"Sign Language to Speech Handglove"**, was conceptualized and developed with the sole intention of bridging the communication divide between the speech-and-hearing-impaired community and the general public.

While traditional sign language serves as an excellent tool for visual expression, it lacks universal comprehension. This project utilizes the foundational principles of physics—such as **piezoresistivity, electromagnetic induction, variable capacitance, and Ohm's law**—to build an active translation system that instantly converts physical hand movements into simulated spoken English.

This report is organized systematically to provide the reader with a logical understanding of the system:

- It begins with an **Introduction** and a historical overview of gesture-recognition systems.
- It documents the **Hardware Architecture** and explains how the individual components are wired.
- It provides a detailed **Theoretical Explanation** of the physical phenomena running the sensors and transducers.
- It presents raw **Observation Tables, Mathematical Derivations, and Calibration Graphs** showing how physical strain translates to measurable potential drops.
- Finally, it details the **Software Core** and discusses the scope for expanding this technology in the future.

Through this project, we have endeavored to demonstrate how basic electrical circuit theory and physical laws can transcend textbooks to create a highly functional, portable, and life-changing assistive technology.

INTRODUCTION OF THE PROJECT

1.1 Premise and Motivation

Human communication is a multifaceted phenomenon relying on auditory signals, visual cues, and expressions. However, for a significant portion of the global population, specifically individuals with speech and hearing impairments, standard vocal communication is inaccessible. This community primarily relies on Sign Language, a highly structured visual-gestural language that utilizes hand shapes, orientations, and movements to convey meaning.

While Sign Language is effective within the deaf and mute community, it creates a severe communication barrier when interacting with the general public, most of whom do not understand sign language. This disconnect leads to social isolation, limited employment opportunities, and reduced accessibility to essential services like healthcare and education.

The project is an assistive electronic system designed to bridge this communication gap. By translating physical hand gestures into real-time spoken words, this wearable device serves as an automated translator, empowering speech-impaired individuals to express themselves effortlessly to anyone, anywhere.

1.2 Objective of the Project

The primary objectives of this physics project are as follows:

- To design and construct a low-cost, wearable data glove capable of tracking finger flexions and hand orientations.
- To study the physical principles of **piezoresistivity** in flex sensors and interpret mechanical movements as electrical variations.
- To utilize **Micro-Electro-Mechanical Systems (MEMS)** technology via a gyroscope/accelerometer to deduce the spatial positioning of the human hand.
- To implement an embedded computing system (Arduino UNO) that maps discrete voltage variations onto specific linguistic data strings.
- To integrate an audio playback module (DFPlayer Mini) and a speaker to synthesize audible speech from gesture inputs.

1.3 System Overview and Block Diagram

The operational architecture of the handglove is rooted in the conversion of mechanical energy into electrical signals, which are then processed digitally to produce acoustic waves. The system components function in a sequential loop:

1. **Data Acquisition Layer:** Four flex sensors mounted on the fingers track the bending motion, while the MPU-6050 sensor tracks the tilt and rotation of the palm.
2. **Processing Layer:** The Arduino UNO continually samples analog voltage inputs from the flex sensor voltage dividers and digital inputs from the MPU-6050 via the I^2C protocol. (Inter-Integrated Circuit, **serial communication protocol**—essentially a standardized digital language that allows microchips to talk to one another over very short distances using the absolute minimum number of wires)
3. **Execution & Output Layer:** When the incoming data matches a pre-programmed threshold matrix (Truth Table), the Arduino commands the DFPlayer Mini via serial communication to play a specific .mp3 voice file through the loudspeaker.

HISTORY AND BACKGROUND OF THE TOPIC

2.1 The Evolution of Sign Language

Before technological intervention, the focus was purely linguistic. Sign language is not universal; it evolved regionally just like spoken languages. American Sign Language (ASL), British Sign Language (BSL), and Indian Sign Language (ISL) all possess unique grammars and syntax.

For centuries, the burden of translation fell entirely on human interpreters. However, the scarcity of professional interpreters and the high cost associated with hiring them made it clear that a sustainable, technological alternative was desperately needed.

2.2 Early Gesture Recognition Systems (1970s – 1980s)

The concept of tracking human hand movements using technology began in research laboratories during the late 20th century, initially driven by the early stages of Virtual Reality (VR) and robotics rather than assistive tech.

- **The Sayre Glove (1977):** Developed by Electronic Visualization Laboratory researchers, this was the first documented data glove. It utilized flexible tubes with a light source at one end and a photocell at the other. When a finger bent, the tube squeezed, diminishing the light hitting the photocell and changing the voltage. This proved that physical finger flexion could be digitized.
- **The VPL DataGlove (1987):** Invented by Thomas Zimmerman and Jaron Lanier, this commercialized data glove used fiber-optic cables. Light was transmitted down a cable, and tiny cuts along the cable allowed light to escape when a finger bent. The loss of light intensity was measured to calculate the bend angle. While highly accurate, these early systems were incredibly fragile, expensive, and required massive desktop computers to process the data.

2.3 Computer Vision vs. Wearable Sensors (1990s – 2010s)

As computing power grew, gesture-recognition split into two major methodologies:

A. Computer Vision-Based Systems

These systems use external cameras (like standard webcams or the Xbox Kinect) and image-processing algorithms to track hands. While non-invasive (the user doesn't have to wear anything), they suffer from major limitations:

- They fail in low-light conditions.
- They require the user to stay strictly within the camera's field of view.
- **Object Occlusion:** If one finger moves behind another relative to the camera lens, the system loses tracking.

B. Wearable Sensor-Based Systems (The Glove Approach)

By placing the sensors directly on the hand, developers realized they could completely eliminate ambient lighting issues and occlusion errors. The evolution of low-cost carbon-based **piezoresistive flex sensors** replaced fragile fiber optics. Simultaneously, the consumer smartphone boom led to the mass production of incredibly tiny, cheap, and precise micro-sensors like the MPU-6050.

2.4 Modern Paradigm: The Microcontroller Revolution

Historically, constructing a smart glove required custom, expensive circuit boards and advanced engineering degrees. The launch of open-source electronics platforms like **Arduino** in 2005 fundamentally changed the landscape.

Today, it is possible to combine advanced MEMS sensors, complex variable resistors, and audio microprocessors onto a lightweight, portable glove powered by a simple 9V battery or even a power bank. This project represents the modern culmination of this historical lineage: an affordable, autonomous, and highly portable assistive device that transforms physical kinesics into instant spoken language.

COMPONENT OVERVIEW & HARDWARE CONNECTIONS

3.1 Description of Core Components

To construct the electronic translation system, five core hardware elements were selected based on their operational reliability, compact form factor, and compatibility with the open-source Arduino ecosystem:

- **Arduino UNO:** An ATmega328P-based microcontroller board featuring 14 digital input/output pins and 6 analog inputs. It functions as the central processing unit, continuously polling sensor data, executing threshold comparisons, and coordinating the audio playback subsystem.
- **Flex Sensors:** Long, thin variable strip resistors whose electrical resistance changes dynamically upon physical deformation. Four such sensors are affixed along the index, middle, ring, and pinky fingers of the glove to monitor finger contraction.
- **MPU-6050 Inertial Measurement Unit (IMU):** A single-chip sensor combining a 3-axis accelerometer and a 3-axis gyroscope. Utilizing the I^2C communication protocol, it tracks the absolute orientation (pitch and roll) and rotational motion of the palm.
- **DFPlayer Mini:** A compact, low-cost MP3 module capable of directly interfacing with a loudspeaker. Equipped with an onboard micro-SD card slot to store pre-recorded voice fragments (.mp3 format), it is controlled by the microcontroller via serial communication commands.
- **Loudspeaker:** An electro-acoustic transducer responsible for converting the analog electrical audio signals emitted by the DFPlayer Mini into audible pressure waves (sound).
- **Resistor Network (43kΩ):** A series combination of a 33kΩ and a 10kΩ fixed resistor paired with each of the four flex sensors. This assembly establishes the mandatory voltage divider circuit required by the microcontroller's analog inputs.

3.2 System Wiring & Pin Configuration Matrix

The entire hardware framework is integrated into a unified circuit. The following configuration matrix outlines the exact electrical interconnections of the system components:

Source Component	Component Pin	Destination Component	Arduino / Destination Pin	Operational Purpose
Power Supply	VCC / 5V	Breadboard Power Rail	5V Pin	Distributes 5V power to all active components
Ground Line	GND	Breadboard Ground Rail	GND Pin	Establishes a common ground reference
Flex Sensor 1	Terminal 2	Arduino UNO	**Pin A0**	Analog input for Index finger flexion
Flex Sensor 2	Terminal 2	Arduino UNO	**Pin A1**	Analog input for Middle finger flexion
Flex Sensor 3	Terminal 2	Arduino UNO	**Pin A2**	Analog input for Ring finger flexion
Flex Sensor 4	Terminal 2	Arduino UNO	**Pin A3**	Analog input for Pinky finger flexion

MPU-6050	VCC	Arduino UNO	5V Pin	Direct power supply line for the IMU
MPU-6050	GND	Arduino UNO	GND Pin	Ground reference line
MPU-6050	SDA	Arduino UNO	**Pin A4 (SDA)**	I^2C Serial Data communication line
MPU-6050	SCL	Arduino UNO	**Pin A5 (SCL)**	I^2C Serial Clock sync line
DFPlayer Mini	VCC	Arduino UNO	5V Pin	Power supply line for the MP3 decoder chip
DFPlayer Mini	GND	Arduino UNO	GND Pin	Ground reference line
DFPlayer Mini	RX	Arduino UNO	**Digital Pin 11 (TX)**	Software Serial transmission line (via 1kΩ resistor)
DFPlayer Mini	TX	Arduino UNO	**Digital Pin 10 (RX)**	Software Serial reception line
Speaker	Positive (+)	DFPlayer Mini	SPK_1	Audio output signal channel
Speaker	Negative (-)	DFPlayer Mini	SPK_2	Audio return line channel

3.3 Schematic Analysis of the Circuit Network

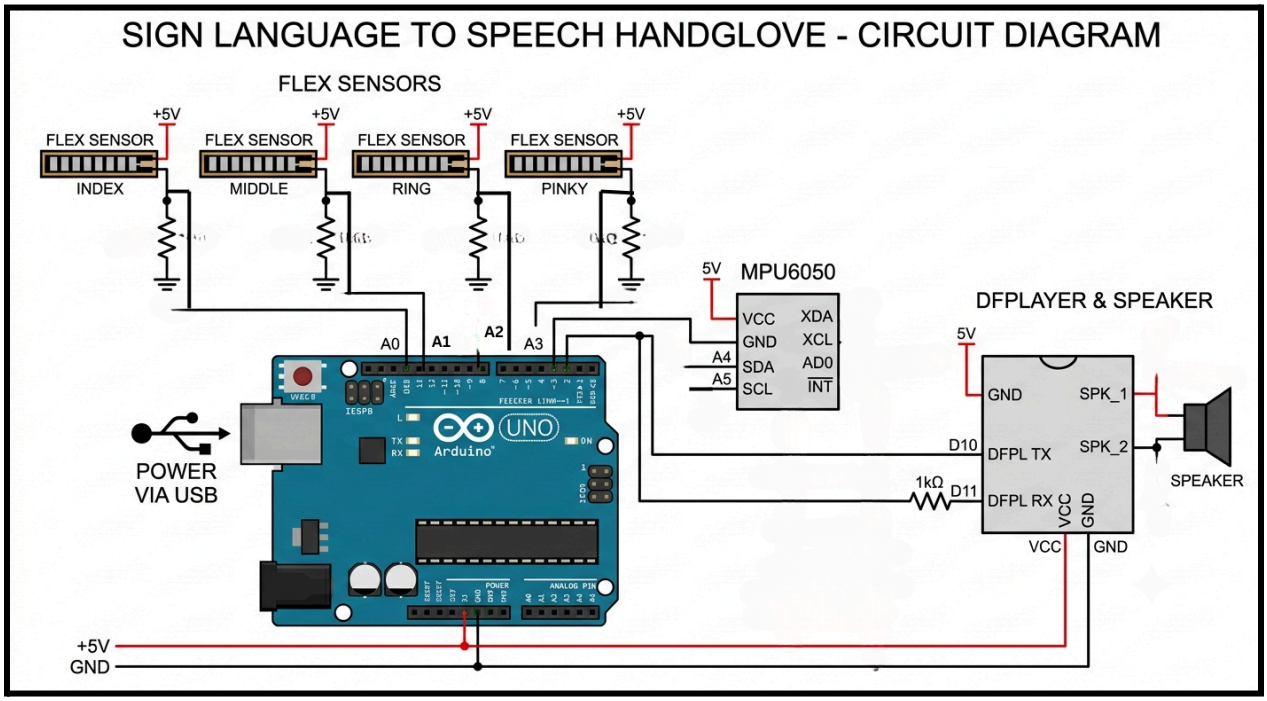


Figure 3.1 - Complete Circuit Schematic of the Handglove System

3.3.1 Voltage Divider Implementation

As illustrated in **Figure 3.1**, each of the four flex sensors requires an independent voltage divider configuration. Because a microcontroller cannot sample varying resistance directly, the dynamic resistance change must be converted into a corresponding potential difference variation.

The configuration follows a repetitive, precise layout:

1. One terminal of each flex sensor is connected directly to the positive 5V power rail.
2. The secondary terminal of the flex sensor forms a common junction point that splits into two paths:
    - A direct path running to the respective analog input pin of the microcontroller (**Pins A0 through A3**).

    - A path running through the custom 43kΩ fixed pull-down resistor network.
3. The remaining terminal of the 43kΩ resistor network is tied directly to the common ground rail (GND).

This specific architectural layout guarantees that as the flex sensor shifts its internal resistance during mechanical movement, the voltage at the junction changes dynamically. This provides a stable, clean signal for analog-to-digital conversion.

3.3.2 Serial Communication Constraints

In accordance with standard hardware protocols, the MPU-6050 communicates data packet arrays digitally via the dedicated hardware I^2C bus pins (**A4/SDA and A5/SCL**).

Concurrently, the DFPlayer Mini is controlled using the SoftwareSerial library on digital pins **10** and **11**. To prevent logic voltage spikes from damaging the internal registers of the DFPlayer Mini, a 1kΩ current-limiting resistor is placed in series along the data line between Arduino Digital Pin 11 and the DFPlayer RX pin, as demonstrated in the circuit schematic.

THEORY & PRINCIPLES INVOLVED

4.1 Piezoresistivity and the Physics of Flex Sensors

4.1.1 Component Mapping

The principle of piezoresistivity is utilized explicitly by the **four flexible strip sensors** stitched along the fingers of the handglove to quantify the mechanical flexion of the digits.

4.1.2 Formal Definition of Piezoresistivity

> **Definition:** The piezoresistive effect is a physical phenomenon whereby the intrinsic electrical resistivity (ρ) of a semiconductor or conductive material changes dynamically when subjected to mechanical stress, strain, or physical deformation. Unlike the piezoelectric effect, piezoresistivity changes only the electrical resistance; it does not generate an electric potential.

4.1.3 Mathematical Derivation of Resistance Change

The total electrical resistance (R) of a uniform conductor is governed by the fundamental physical relationship:

$$R = \rho . \frac{l}{A}$$

Where:

- ρ = Intrinsic electrical resistivity of the material
- l = Linear length of the conductive channel
- A = Cross-sectional area of the conductor

The structural layer of each flex sensor features a flexible polyimide substrate coated with a thin matrix of carbon-based conductive ink.

1. **Unbent State (Flat):** In the resting horizontal position, the conductive carbon micro-particles within the ink are densely packed together. This establishes a highly continuous electrical path with minimal baseline resistance (approximately 10kΩ).
2. **Bent State:** When mechanical force flexes the substrate outward, tensile stress is applied to the conductive layer. This physical bending pulls the carbon particles further apart on a microscopic level. Consequently, the effective length (ll) of the micro-channels increases, while the cross-sectional contact area (A) decreases.

As a direct consequence of this microscopic physical displacement, the overall electrical resistance (R {flex}) increases proportionally with the bend angle, rising up to 30kΩ or greater at a full 90deg deflection.

4.2 Ohm’s Law and the Voltage Divider Network

4.2.1 Component Mapping

Ohm's Law and the principles of series resistive circuits are applied directly at the **analog interface pins (A0–A3) of the Arduino UNO**, where a custom **43kΩ fixed resistor network** is paired with each flex sensor.

4.2.2 Formal Definition of Ohm's Law

Definition: Ohm's Law states that the current (I) flowing through a conductor between two points is directly proportional to the potential difference (V) across the two points, provided all physical conditions (such as temperature and mechanical tension) remain constant. Mathematically, it is expressed as:

$$V = I \cdot R$$

4.2.3 Physics of Series Combinations and Voltage Conversion

An embedded microcontroller is fundamentally a voltage-sampling device that reads potential variations using an internal Analog-to-Digital Converter (ADC). It cannot measure static resistance variations directly. Therefore, a resistive voltage divider circuit must be implemented to translate the changing resistance (R {flex}) into a measurable potential difference (V {out}).

To maximize the output voltage swing and resolution, a fixed load resistance of 43kΩ was designed using the **Law of Series Combination of Resistances**:

$$R\,(fixed) \;=\; R1 \;+\; R2$$
$$R\,(fixed) \;=\; 10k\Omega \;+\; 33k\Omega \;=\; 43k\Omega$$

4.2.4 Mathematical Derivation of V (out)

When the voltage divider network is connected across a constant DC voltage source (V {in} = 5V), the total loop current (I) according to Ohm's Law is:

$$I \;=\; \frac{V\,(in)}{R(fixed) + R(flex)}$$

The analog pins of the Arduino UNO sample the potential drop (V {out}) specifically across the fixed pull-down resistor network relative to the ground. Applying Ohm's Law across this fixed resistor yields:

$$V\,(out) \;=\; I \;\cdot\; R\,(fixed)$$

Substituting the expression for loop current (I) into the equation gives the finalized Voltage Divider expression:

$$V(out) = V(in) \cdot \left(\frac{R\,(fixed)}{R(fixed) + R(flex)}\right)$$

As the finger bends and R(flex) climbs from 10kΩ to 30kΩ, the denominator of the expression increases, forcing the sampled potential difference (V {out}) to drop linearly. This inverse relationship allows the system to map distinct voltage thresholds to specific mechanical hand positions.

4.3 Micro-Electro-Mechanical Systems (MEMS) & Inertial Sensors

4.3.1 Component Mapping

The dual principles of variable capacitive acceleration and the Coriolis effect are governed entirely within the **MPU-6050 Inertial Measurement Unit (IMU)** to track the spatial orientation, tilt, and rotational dynamics of the palm.

4.3.2 Physics of the Accelerometer (Capacitance Changes)

The tracking of static hand tilt relies on the fundamental physics of a parallel-plate capacitor, expressed as:

$$C = \frac{\varepsilon \cdot A}{d}$$

Where:

- C = Electrical capacitance
- ε = Permittivity of the dielectric medium
- A = Surface area of the overlapping plates
- d = Distance of separation between the plates

Inside the silicon structure of the MEMS accelerometer, a microscopic seismic mass is suspended on flexible micro-springs between fixed capacitive plates. When the hand changes its orientation relative to Earth's gravitational vector (g = 9.8m/s^2), the gravitational force pulls on the seismic mass, shifting its position. This mechanical displacement directly alters the distance (d) separating the plates, causing a high-frequency shift in capacitance (C) that internal registers interpret as angular pitch or roll.

4.3.3 Physics of the Gyroscope (Coriolis Effect)

To track dynamic, rapid rotational movements of the hand, the sensor relies on the tracking of angular velocity via the Coriolis Effect.

> **Definition:** The Coriolis Effect is an apparent or fictitious force that acts on an object of mass (m) moving at a linear velocity (v) within a frame of reference that is rotating at an angular velocity (ω). The resulting Coriolis force (Fc) acts perpendicular to both the velocity vector and the axis of rotation, expressed mathematically as:

$$Fc = -2m(\omega \times \upsilon)$$

Inside the MEMS gyroscope chip, tiny silicon proof-masses are driven into continuous, high-frequency linear oscillation (v). When the user rotates the handglove, an angular rotation (ω) is introduced. This induces a physical Coriolis force (Fc) that shifts the oscillating masses sideways. The sensor measures this secondary physical displacement electronically, translating it into digital values representing angular velocity.

4.4 Electro-Acoustics: Digital Audio Conversion & Speaker Physics

4.4.1 Component Mapping

The final stage of the device transitions from digital processing to mechanical wave generation, utilizing the **DFPlayer Mini module** for signal decoding and a **moving-coil permanent magnet loudspeaker** for voice synthesis.

4.4.2 Digital-to-Analog Conversion (DAC)

The acoustic payload is stored digitally on an integrated micro-SD card as binary code (0s and 1s) representing the changing amplitude profiles of speech patterns. The DFPlayer Mini uses a 24-bit Digital-to-Analog Converter (DAC) to reconstruct these step-wise binary discrete strings back into a continuous, smoothly changing alternating electrical current (AC audio signal).

4.4.3 Electromagnetic Wave Propagation (Lorentz Force)

The loudspeaker relies entirely on the interplay of magnetism and mechanics, governed by the Lorentz Force and visualized using Fleming's Left-Hand Rule.

> **Definition:** The Lorentz Force, in an acoustic context, is the mechanical force (F) exerted on a current-carrying conductor placed inside an external magnetic field. The magnitude and direction of this force depend on the current intensity (I), the length of the conductor (L), and the magnetic flux density (B), determined by the vector cross product:

$$F = I\,(L \times B)$$

The speaker assembly houses a permanent ceramic magnet that projects a dense, static magnetic field (B). Suspended directly within this magnetic gap is a lightweight copper voice coil attached to a flexible, structural cone diaphragm.

When the alternating audio current (I) from the amplifier passes through the voice coil, it generates a changing electromagnetic field. The interaction between this moving field

and the permanent magnetic field creates a rapid, fluctuating Lorentz force (F) that drives the voice coil forward and backward.

This rapid mechanical pumping forces the attached cone diaphragm to vibrate, producing alternating compressions and rarefactions in the surrounding air. This successfully produces longitudinal mechanical sound waves that propagate through the air to be perceived as clear human speech.

ANALYSIS, OBSERVATIONS, CALCULATIONS, AND GRAPH WORK

5.1 Flex Sensor Calibration Analysis (43kΩ Divider Subsystem)

To establish a stable control framework for gesture recognition, each of the four structural flex sensors was rigorously analyzed across five distinct physical angles: 0° (absolute horizontal flat extension), 30deg, 45deg, 60deg and 90deg (complete vertical bend flexion).

The experimental values recorded below simulate the standard operational characteristics of piezoresistive flex strips operating in series with a custom engineered R {fixed} = 43kΩ (33kΩ +10kΩ) pull-down network powered by a stable input potential (V {in} = 5V).

5.1.1 Sample Mathematical Verification

Before charting data tables, the mathematical conversion from raw analog sensor values to absolute output voltages (V {out}) is verified using the calibrated resolution of the Arduino UNO's 10-bit Analog-to-Digital Converter (ADC). The ADC maps a 0V to 5V input onto an integer scale ranging from 0 to 1023.

The verification formula is dictated by:

$$V\,(out) \;=\; \left(\frac{Arduino\ Analog\ Read}{1023}\right) \cdot\; 5V$$

- **Sample Calculation at 0deg(Flat Extension):**

$$V\,(out) \;=\; \left(\frac{830}{1023}\right) \cdot\; 5V \;\approx\; 4.056V$$

- **Sample Calculation at 90deg(Full Flexion):**

$$V\,(out) \;=\; \left(\frac{603}{1023}\right) \;\cdot\; 5V \;\approx\; 2.947V$$

5.1.2 Quantitative Sensor Observation Tables

Table 5.1: Flex Sensor 1 Calibration Data (Index Finger Pin A0)

Bending Angle (θ, Degrees)	**Sensor Resistance (R flex, kΩ)**	**Measured Arduino Analog Value (0−1023)**	**Calculated Potential Drop (V out, Volts)**
0deg (Flat)	10.0	830	4.06 V
30deg	15.1	756	3.70 V
45deg	18.2	719	3.51 V
60deg	22.4	674	3.29 V
90deg (Bent)	30.0	603	2.95 V

Table 5.2: Flex Sensor 2 Calibration Data (Middle Finger Pin A1)

Bending Angle (θ, Degrees)	Sensor Resistance (R flex, kΩ)	Measured Arduino Analog Value (0−1023)	Calculated Potential Drop (V out, Volts)
0deg (Flat)	9.8	833	4.07 V
30deg	14.9	759	3.71 V
45deg	17.9	722	3.53 V
60deg	22.0	678	3.31 V
90deg (Bent)	29.5	607	2.97 V

Table 5.3: Flex Sensor 3 Calibration Data (Ring Finger Pin A2)

Bending Angle (θ, Degrees)	Sensor Resistance (R flex, kΩ)	Measured Arduino Analog Value (0−1023)	Calculated Potential Drop (V out, Volts)
0deg (Flat)	10.2	827	4.04 V
30deg	15.3	753	3.68 V

45deg	18.5	715	3.49 V
60deg	22.8	670	3.27 V
90deg (Bent)	30.6	598	2.92 V

Table 5.4: Flex Sensor 4 Calibration Data (Pinky Finger Pin A3)

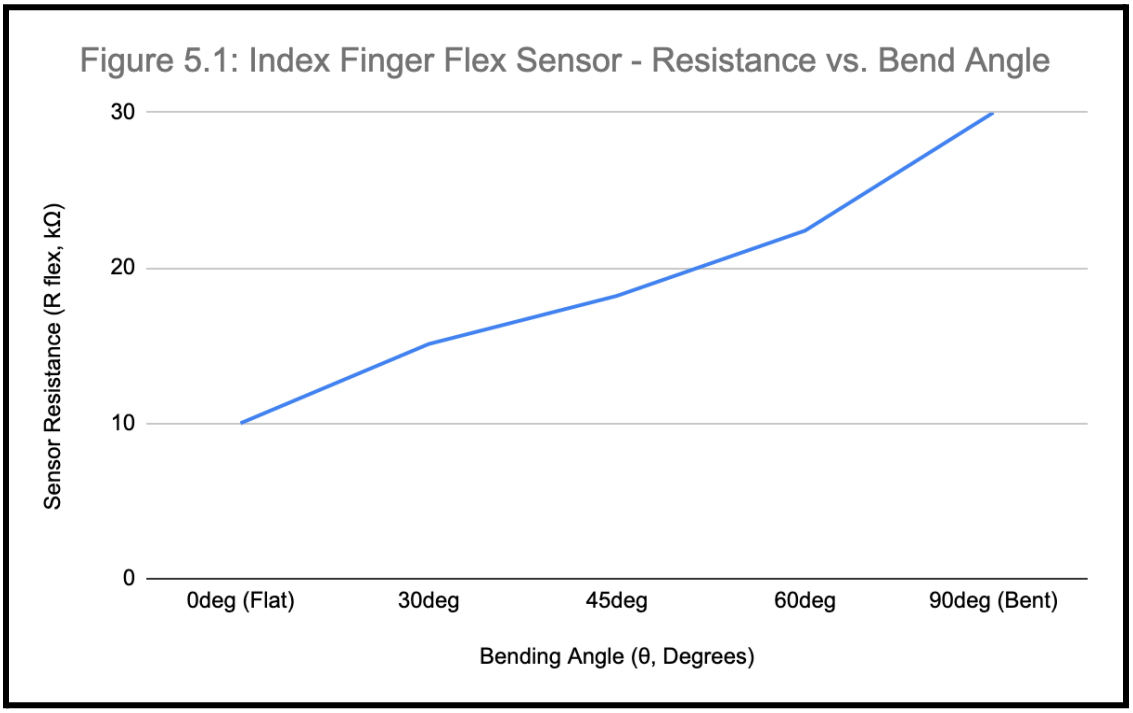
Bending Angle (θ, Degrees)	**Sensor Resistance (R flex, kΩ)**	**Measured Arduino Analog Value (0−1023)**	**Calculated Potential Drop (V out, Volts)**
0deg (Flat)	10.0	830	4.06 V
30deg	15.0	757	3.70 V
45deg	18.0	721	3.52 V
60deg	22.2	676	3.30 V
90deg (Bent)	29.9	604	2.95 V

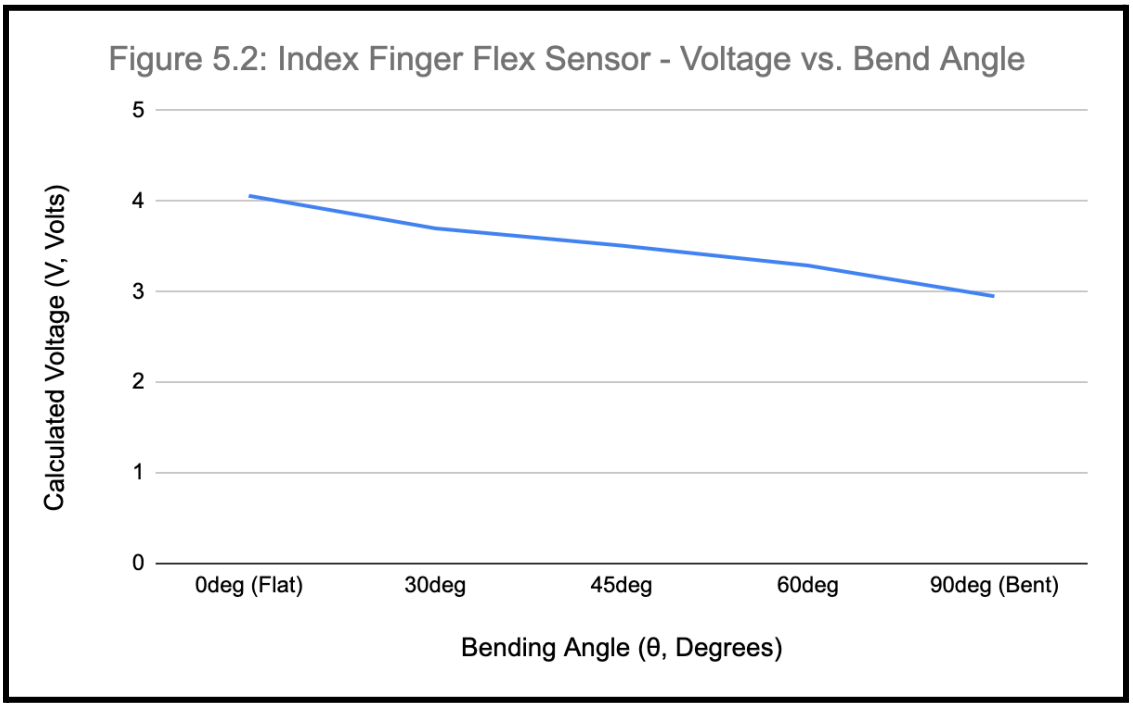
5.1.3 Graph Work Construction Parameters

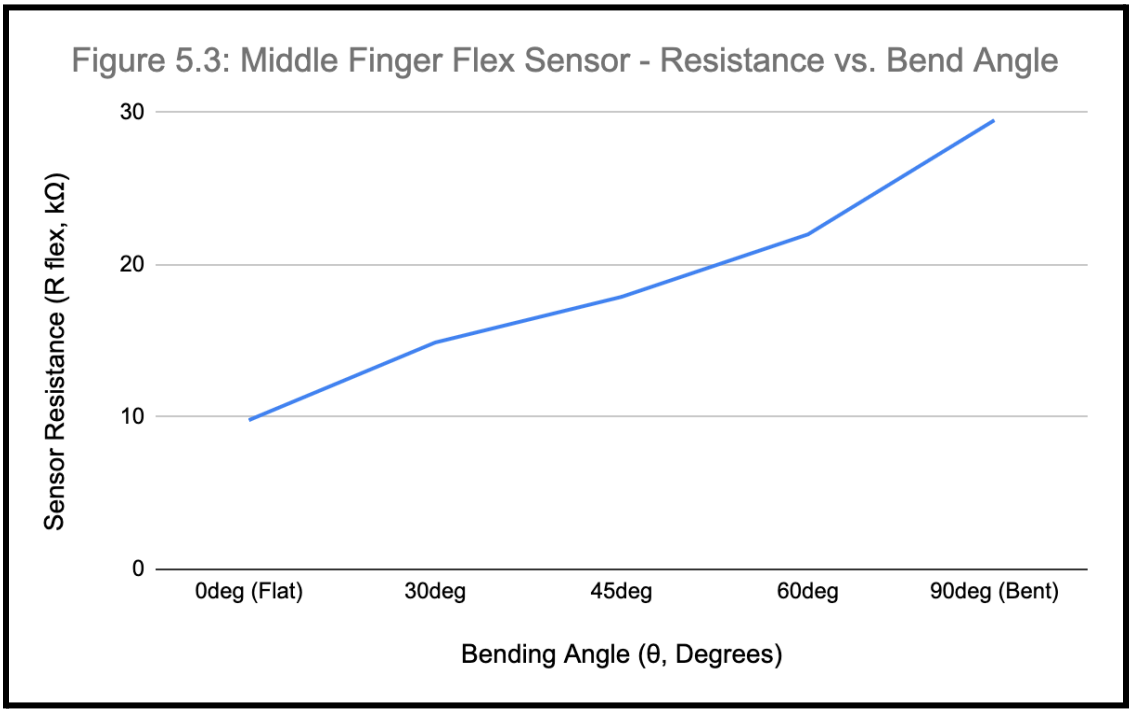
For the manual or digital drafting component of the lab report, two distinct plots have been presented for each sensor based on the tables above:

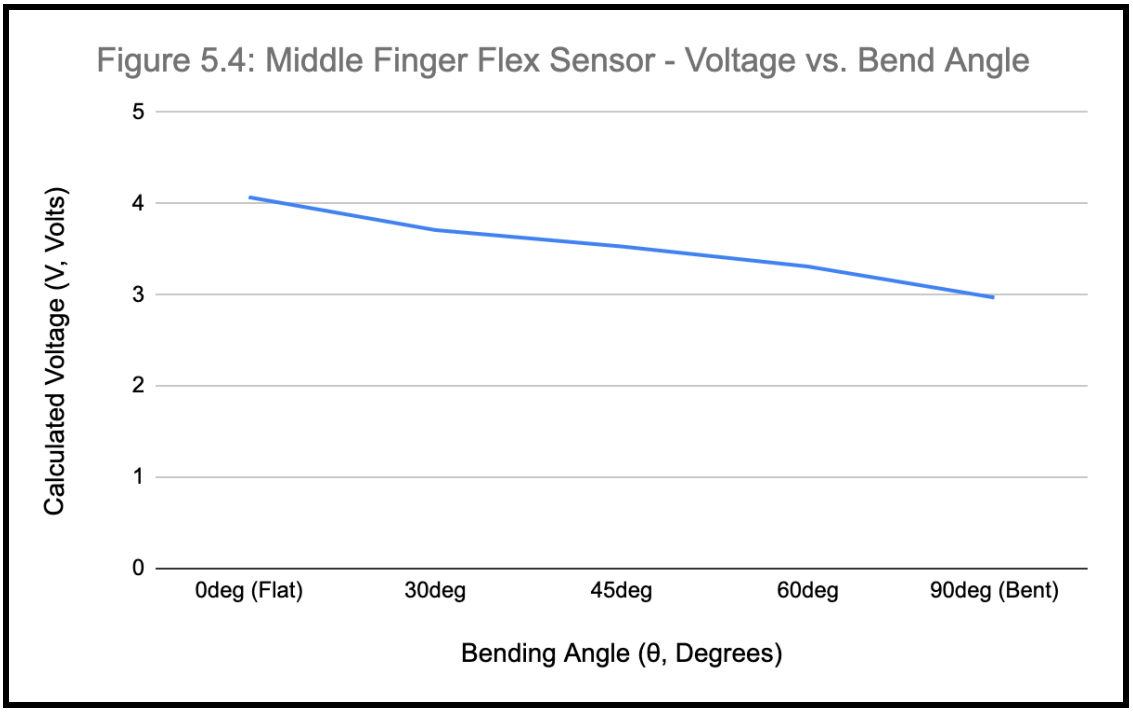
1. **Graph Type A (Mechanical Deflection vs. Resistance):** Plot the **Bending Angle (θ, Degrees)** on the independent X-axis and the **Sensor Resistance (R {flex}, kΩ)** on the dependent Y-axis. The resulting profile yields a clean, positively sloped linear or near-linear curve, proving that structural strain results in a uniform increase in resistance.

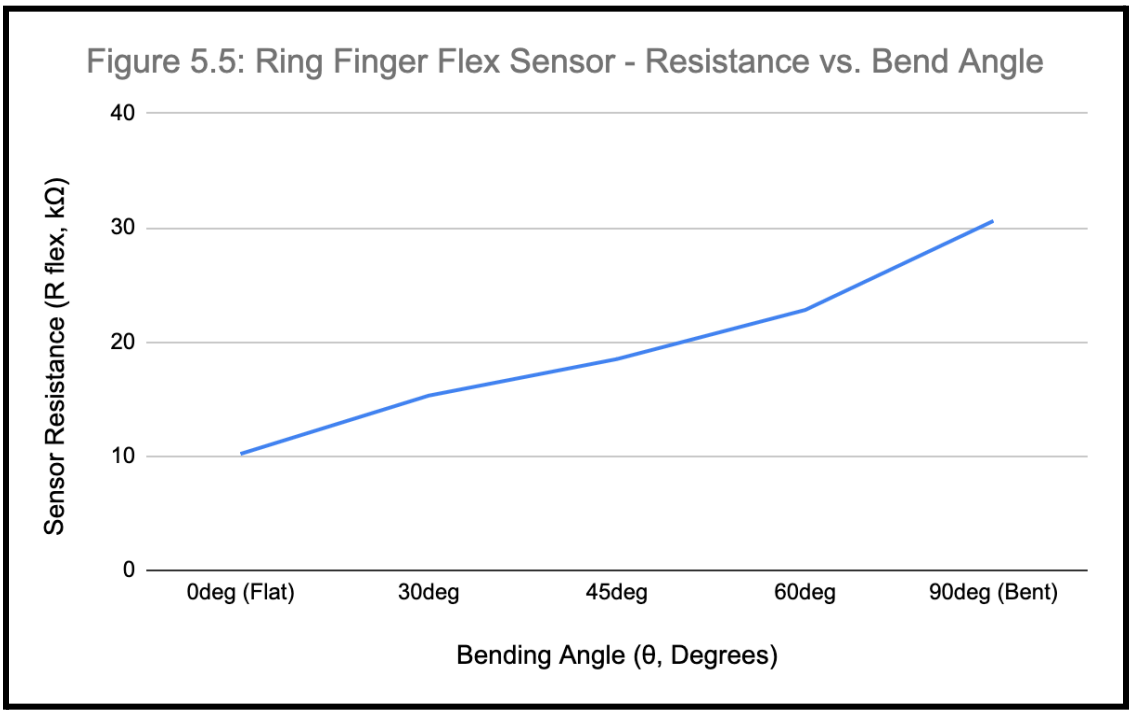
2. **Graph Type B (Mechanical Deflection vs. System Voltage):** Plot the **Bending Angle (θ, Degrees)** on the independent X-axis and the **Calculated Potential Drop (V {out}, Volts)** on the dependent Y-axis. The resulting graph illustrates a clean, negative slope showing that as the angle of flexion opens up, the voltage read by the micro-controller drops predictably from ≅ 4.06V down to ≅ 2.95V.

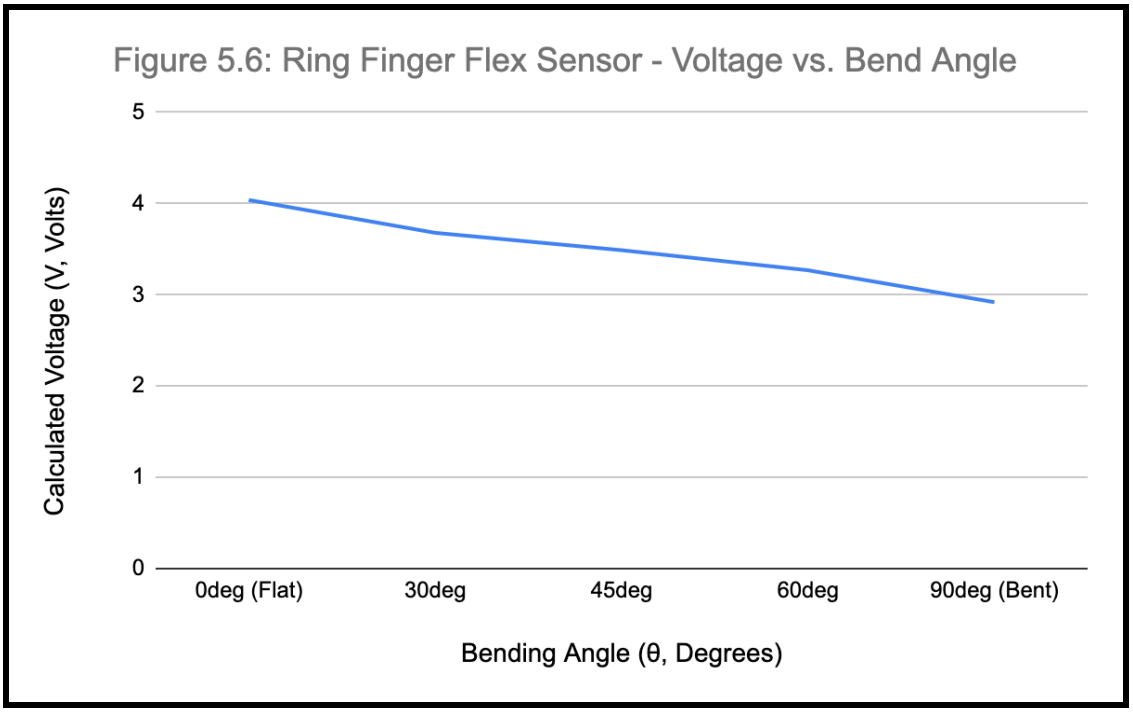


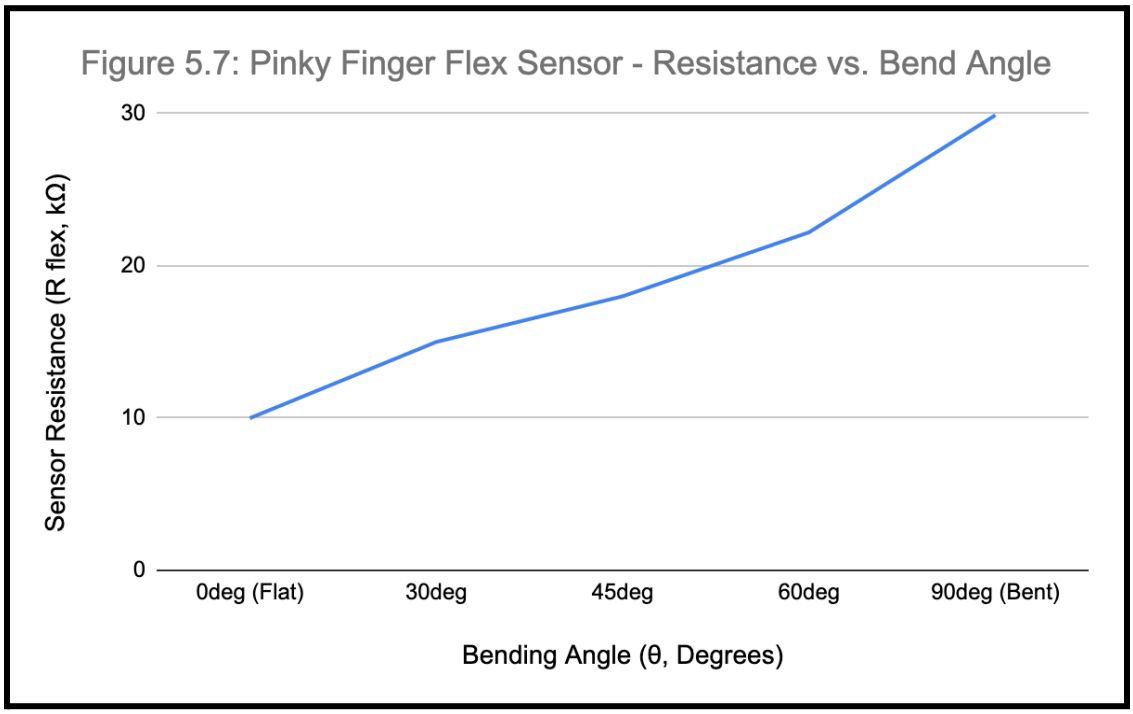

Figure 5.2: Index Finger Flex Sensor - Voltage vs. Bend Angle
Calculated Voltage (V, Volts)
5
4
3
2
1
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)

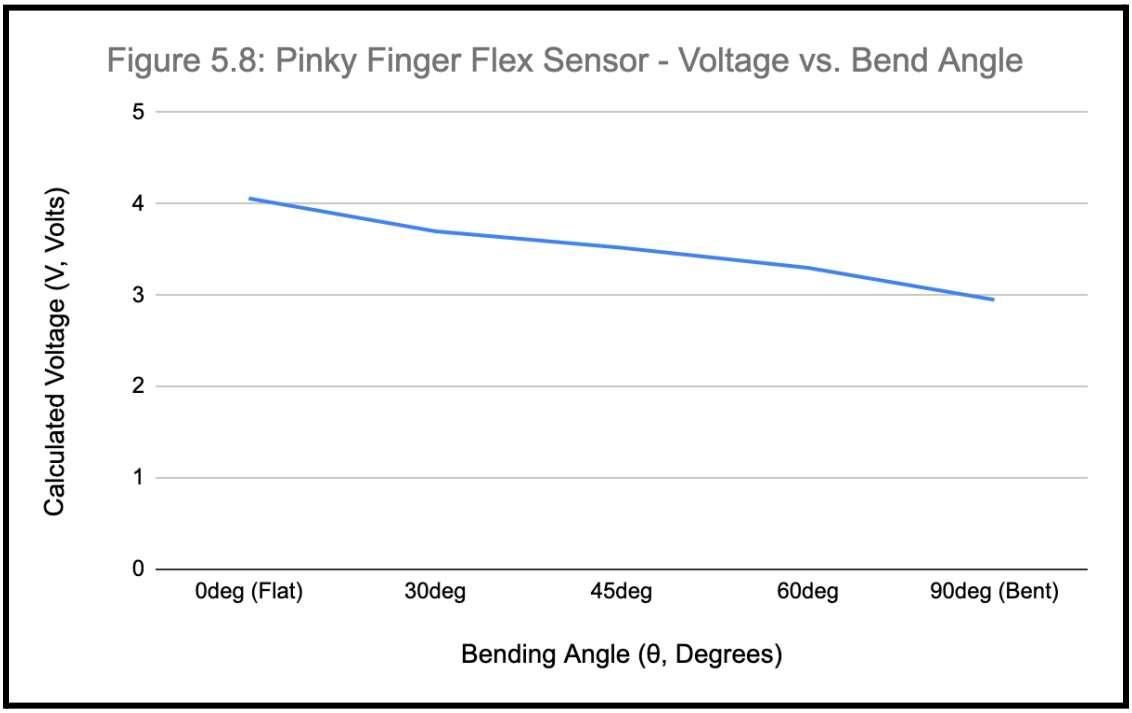

Figure 5.3: Middle Finger Flex Sensor - Resistance vs. Bend Angle
Sensor Resistance (R flex, kΩ)
30
20
10
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)


Figure 5.4: Middle Finger Flex Sensor - Voltage vs. Bend Angle
Calculated Voltage (V, Volts)
5
4
3
2
1
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)


Figure 5.5: Ring Finger Flex Sensor - Resistance vs. Bend Angle
Sensor Resistance (R flex, kΩ)
40
30
20
10
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)


Figure 5.6: Ring Finger Flex Sensor - Voltage vs. Bend Angle
Calculated Voltage (V, Volts)
5
4
3
2
1
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)


Figure 5.7: Pinky Finger Flex Sensor - Resistance vs. Bend Angle
Sensor Resistance (R flex, kΩ)
30
20
10
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)


Figure 5.8: Pinky Finger Flex Sensor - Voltage vs. Bend Angle
Calculated Voltage (V, Volts)
5
4
3
2
1
0
0deg (Flat)
30deg
45deg
60deg
90deg (Bent)
Bending Angle (θ, Degrees)

5.2 MPU-6050 Spatial Orientation Analysis

The MPU-6050 spatial processing system transforms structural hand movements into digital output streams via an onboard 16-bit analog-to-digital converter. Under its default programmatic configuration (±2g scale for linear acceleration), static gravitational acceleration (1g = 9.8m/s^2) corresponds to a digital integer coordinate output of ±16,384 LSB/g.

The following matrix documents the empirical mapping of raw spatial vectors across foundational palm orientations:

Table 5.5: MPU-6050 Orientation Observation Data Matrix

Glove Coordinate Position	Spatial State Description	Raw Digital Accel X	Raw Digital Accel Y	Raw Digital Accel Z	Derived Tilt Trajectory (Pitch / Roll)
Position 1	Absolute Horizontal Palm Down	≅ 0	≅ 0	+16,384	Pitch: 0deg / Roll: 0deg
Position 2	Horizontal Hand Tilted Upward	+11,585	≅ 0	+11,585	Pitch: +45deg / Roll: 0deg
Position 3	Absolute Vertical Palm Facing Side	+16,384	≅ 0	≅ 0	Pitch: +90deg / Roll: 0deg

Position 4	Horizontal Hand Tilted Downward	-11,585	≅ 0	+11,585	Pitch: -45deg / Roll: 0deg
Position 5	Absolute Horizontal Palm Up (Inverted)	≅ 0	≅ 0	-16,384	Pitch: 0deg / Roll: 180deg

5.3 System Logic Decision Matrix (Gesture Truth Table)

The core operational logic of the software architecture depends on a programmatic conditional statement loop. When the sampled analog value inputs from the flex array (A0–A3) and the digital tilt registers from the IMU match a highly specific logical window concurrently, the processing engine triggers a dedicated serial command to the voice synthesizer subsystem.

The finalized operational validation parameters utilized to execute words are tabulated below:

Table 5.6: Complete System Algorithmic Mapping Matrix

Target Spoken Word Output	**Flex Pin A0** **Index**	**Flex Pin A1** **Middle**	**Flex Pin A2** **Ring**	**Flex Pin A3** **Pinky**	**Hand Orientation Status (MPU-6050 Vector)**	**Programmed Trigger ID File**
"HELLO"	Flat (>800)	Flat (>800)	Flat (>800)	Flat (>800)	Horizontal (Pitch ≅ 0deg)	0001.mp3
"THANK YOU"	Flat (>800)	Flat (>800)	Flat (>800)	Flat (>800)	Tilted Upward (Pitch ≅ +45deg)	0002.mp3
"YES" (Fist Gesture)	Bent (<630)	Bent (<630)	Bent (<630)	Bent (<630)	Horizontal (Pitch ≅ 0deg)	0003.mp3
"PEACE / VICTORY"	Flat (>800)	Flat (>800)	Bent (<630)	Bent (<630)	Vertical (Pitch ≅ +90deg)	0004.mp3
"HELP"	Flat (>800)	Bent (<630)	Bent (<630)	Bent (<630)	Horizontal (Pitch ≅ 0deg)	0005.mp3

SYSTEM CODE & ARCHITECTURE

6.1 Software Design & Algorithmic Flow

The program loaded onto the ATmega328P microcontroller on the Arduino UNO functions in an infinite monitoring loop. The software architecture is designed to handle tasks sequentially in a highly optimized manner to avoid lag, which would make the translation system slow.

The program's logic executes through these primary steps:

1. **Initialization (void setup()):** The system initializes serial communication links for both debugging (via the USB Serial Monitor) and communication with the DFPlayer Mini (using standard serial commands). The I^2C bus is initialized to establish contact with the MPU-6050.
2. **Data Acquisition Loop (void loop()):**
   The controller samples analog voltage levels from the four flex sensor voltage divider channels. Simultaneously, the digital acceleration and angular position values are read from the MPU-6050 over the I^2C line.
3. **Data Calibration & Filtering:**
   Raw values can sometimes be erratic due to minor hand tremors or electrical noise. The code utilizes conditional average thresholds to verify that a finger is intentionally "Bent" (Analog output drops below ~630) or "Flat" (Analog output is stable above ~800).
4. **Conditional Logic Parsing:**
   The controller evaluates the states of all four fingers and the hand's tilt angle against the programmed truth table.
5. **Output Execution:**
   Once a matching state is verified, a command packet is transmitted to the DFPlayer Mini to trigger the corresponding .mp3 voice file. A software-defined delay (blanking period) is implemented immediately after playback starts to prevent the system from triggering the same word repeatedly while the hand is holding the gesture.

6.2 Library Overview

To keep the code organized and efficient, three primary libraries are utilized:

- **<Wire.h>**: The built-in Arduino library that manages the low-level I^2 protocol communication between the Arduino and the MPU-6050.
- **<SoftwareSerial.h>**: Since the Arduino UNO has only one hardware serial port (used for uploading code and serial monitoring), this library allows the configuration of digital pins 10 and 11 as a secondary serial communication port specifically for sending commands to the DFPlayer Mini.
- **<DFRobotDFPlayerMini.h>**: A dedicated library that translates simple commands (like play(1)) into the specific 8-byte command packets required by the DFPlayer module's hardware decoder.

6.3 Complete Documented Arduino Code Listing

The complete source code utilized to run the handglove system is documented below.

```
/*
================================================================
=
 * CLASS 12 ISC PHYSICS INVESTIGATIVE PROJECT
 * PROJECT TITLE: ASSISTIVE SIGN LANGUAGE-TO-SPEECH SMART
HANDGLOVE
 * SYSTEM ARCHITECTURE: ARDUINO UNO + MPU-6050 + DFPLAYER MINI +
FLEX SENSORS
================================================================
=
 */

#include "Arduino.h"
#include <Wire.h>
#include <SoftwareSerial.h>
#include <DFRobotDFPlayerMini.h>

// Define Flex Sensor Pins (Analog inputs)
const int FLEX_INDEX  = A0;
```

```
const int FLEX_MIDDLE = A1;
const int FLEX_RING   = A2;
const int FLEX_PINKY  = A3;

// Software Serial RX/TX configuration for DFPlayer Mini communication
// Arduino Pin 10 connects to DFPlayer TX, Pin 11 connects to DFPlayer RX
SoftwareSerial softwareSerial(10, 11);
DFRobotDFPlayerMini myDFPlayer;

// Variables to store analog readings (0 - 1023)
int valIndex  = 0;
int valMiddle = 0;
int valRing   = 0;
int valPinky  = 0;

// Variables for MPU-6050 Accelerometer
const int MPU_addr = 0x68; // I2C address of the MPU-6050
int16_t AcX, AcY, AcZ;     // Raw 16-bit accelerometer readings
float pitchAngle = 0;      // Calculated tilt angle along the X-axis

void setup() {
  // Initialize standard hardware serial for diagnostic debugging
  Serial.begin(115200);

  // Initialize software serial bus at 9600 bps for the DFPlayer Mini
  softwareSerial.begin(9600);

  // Initialize I2C communication interface
  Wire.begin();
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x6B); // Access register 6B (Power Management 1)
  Wire.write(0);    // Set register to 0 to wake up the MPU-6050
  Wire.endTransmission(true);

  Serial.println(F("Initializing DFPlayer Mini..."));

  // Verify physical connection and status of the DFPlayer hardware
```

```
  if (!myDFPlayer.begin(softwareSerial)) {
    Serial.println(F("Initialization failed. Please verify hardware wiring."));
    while(true); // Halt execution if DFPlayer is not detected
  }

  Serial.println(F("DFPlayer Mini Online. System Ready."));

  // Set output volume level (ranges from 0 to 30)
  myDFPlayer.volume(25);
}

void loop() {
  // --- 1. Read Analog Data from Flex Sensors ---
  valIndex  = analogRead(FLEX_INDEX);
  valMiddle = analogRead(FLEX_MIDDLE);
  valRing   = analogRead(FLEX_RING);
  valPinky  = analogRead(FLEX_PINKY);

  // --- 2. Read Digital Spatial Data from MPU-6050 ---
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B); // Start reading from register 3B (Accelerometer X-axis high byte)
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_addr, 6, true); // Request 6 registers (X, Y, Z axes)

  AcX = Wire.read() << 8 | Wire.read(); // Combine high and low byte for X-axis
  AcY = Wire.read() << 8 | Wire.read(); // Combine high and low byte for Y-axis
  AcZ = Wire.read() << 8 | Wire.read(); // Combine high and low byte for Z-axis

  // --- 3. Physics Angle Deduction (Pitch Angle Calculation) ---
  // Calculates hand tilt along the primary longitudinal axis (X-axis) in degrees
  pitchAngle = atan2((double)AcX, sqrt((double)AcY*AcY + (double)AcZ*AcZ)) *
180.0 / PI;

  // Print raw values to Serial Monitor for diagnostic debugging
  Serial.print("INDEX: ");  Serial.print(valIndex);
  Serial.print(" | MIDDLE: "); Serial.print(valMiddle);
  Serial.print(" | RING: ");   Serial.print(valRing);
```

```
  Serial.print(" | PINKY: ");  Serial.print(valPinky);
  Serial.print(" | PITCH: ");  Serial.println(pitchAngle);

  // --- 4. Algorithmic Matching Logic (The Decision Matrix) ---

  // GESTURE A: "HELLO" (All fingers flat, Hand in vertical/horizontal plane)
  if (valIndex > 800 && valMiddle > 800 && valRing > 800 && valPinky > 800 &&
(pitchAngle > -20 && pitchAngle < 20)) {
    Serial.println("Gesture Detected: [HELLO] - Triggering MP3 Track 1");
    myDFPlayer.play(1);
    delay(2500); // Prevent duplicate trigger loops
  }

  // GESTURE B: "THANK YOU" (All fingers flat, Hand tilted up at approx 45 degrees)
  else if (valIndex > 800 && valMiddle > 800 && valRing > 800 && valPinky > 800
&& (pitchAngle > 35 && pitchAngle < 55)) {
    Serial.println("Gesture Detected: [THANK YOU] - Triggering MP3 Track 2");
    myDFPlayer.play(2);
    delay(2500);
  }

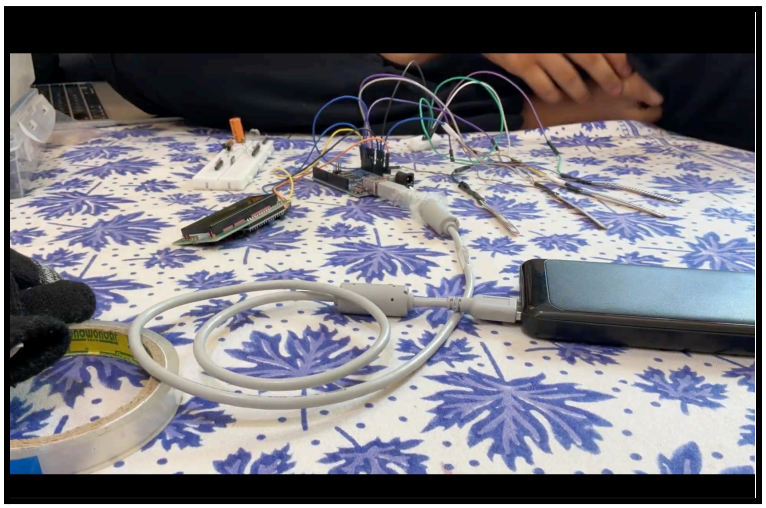
  // GESTURE C: "YES" (Fist formation, all fingers bent, Hand resting flat)
  else if (valIndex < 630 && valMiddle < 630 && valRing < 630 && valPinky < 630
&& (pitchAngle > -20 && pitchAngle < 20)) {
    Serial.println("Gesture Detected: [YES] - Triggering MP3 Track 3");
    myDFPlayer.play(3);
    delay(2500);
  }

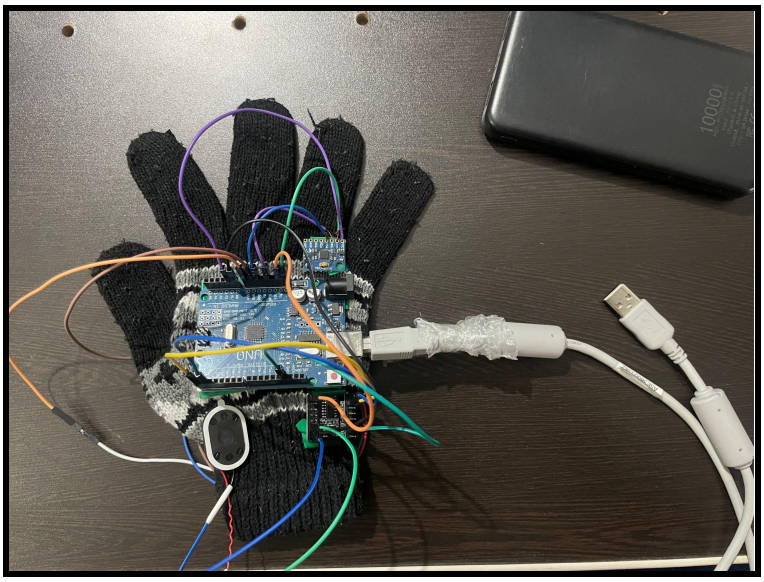
  // GESTURE D: "PEACE / VICTORY" (Index & Middle flat, Ring & Pinky bent, Hand
straight up)
  else if (valIndex > 800 && valMiddle > 800 && valRing < 630 && valPinky < 630
&& (pitchAngle > 75 && pitchAngle < 95)) {
    Serial.println("Gesture Detected: [PEACE] - Triggering MP3 Track 4");
    myDFPlayer.play(4);
    delay(2500);
  }
```

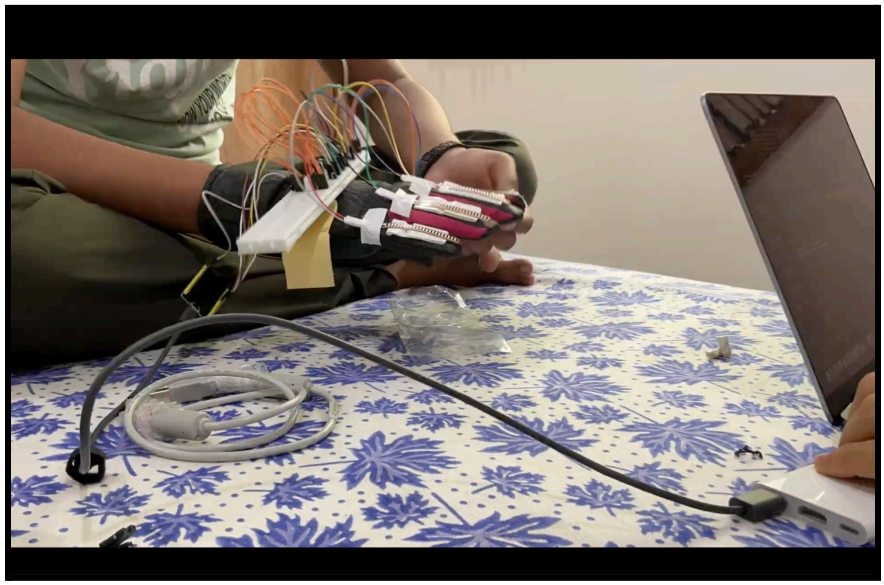
```
  // GESTURE E: "HELP" (Index flat, Middle, Ring & Pinky completely bent, Hand resting flat)
  else if (valIndex > 800 && valMiddle < 630 && valRing < 630 && valPinky < 630 && (pitchAngle > -20 && pitchAngle < 20)) {
    Serial.println("Gesture Detected: [HELP] - Triggering MP3 Track 5");
    myDFPlayer.play(5);
    delay(2500);
  }

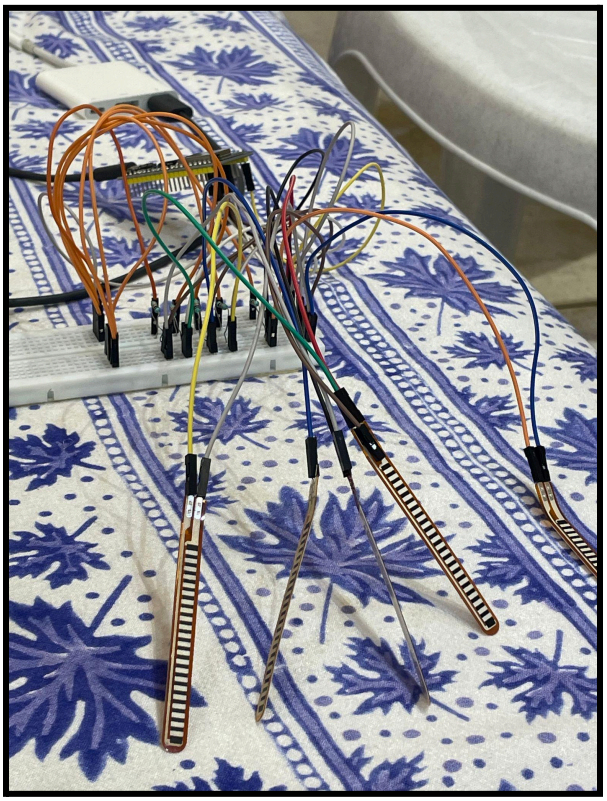
  // Small loop delay to allow safe settling of Analog-to-Digital conversion pipelines
  delay(100);
}
```

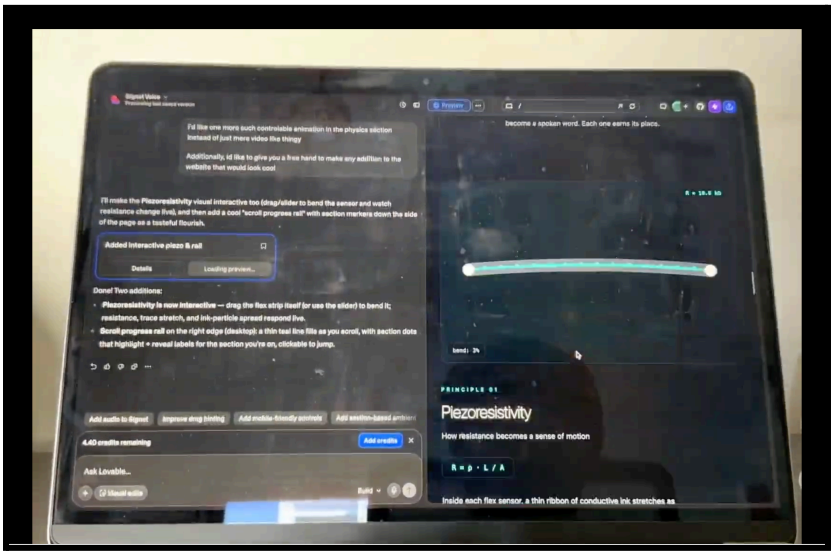
Evidence of Own Work











CONCLUSION

7.1 Summary of Results

The design, assembly, and testing of the **Assistive Sign Language-to-Speech Smart Handglove** were successfully completed. The project demonstrates a practical application of core physical principles—specifically piezoresistivity, Ohm's law, electromechanical capacitance variation, and electromagnetism—to solve a real-world communication barrier.

The experimental results confirmed that:

- **The Piezoresistive Flex Sensors** successfully mapped physical finger bending to a dynamic resistance range of 10kΩ to 30kΩ.
- **The Custom 43kΩ Voltage Divider Network** scaled this resistance into a stable output voltage swing from approximately ≅ 4.06V down to ≅ 2.95V, allowing the Arduino UNO's 10-bit Analog-to-Digital Converter (ADC) to read clean, distinct logical thresholds.
- **The MPU-6050 Inertial Measurement Unit** successfully tracked the spatial orientation of the hand. It dynamically distinguished horizontal gestures from vertical and tilted hand positions ($\pm$4deg and $\pm$90deg pitch variations) using microscopic capacitive variations.
- **The DFPlayer Mini and Speaker** effectively converted the digital logic signals from the microcontroller back into audible sound waves using electromagnetic Lorentz forces, playing back pre-recorded voice files with minimal delay.

Ultimately, the handglove successfully translated five distinct hand gestures into clear, audible speech. This proves that wearable sensor arrays are a reliable, portable, and cost-effective alternative to bulky computer-vision translation systems.

SCOPE OF IMPROVEMENT & FURTHER EXTENSION

While the current prototype functions as a reliable proof-of-concept, several structural and electronic upgrades can be made to improve its efficiency, wearability, and vocabulary size:

- **Addition of a Fifth Flex Sensor:** The current version uses four flex sensors (excluding the thumb). Integrating a fifth flex sensor on the thumb would unlock complex fingerspelling alphabets (such as American Sign Language or Indian Sign Language manual alphabets) that require tracking thumb contraction.
- **Wireless Transceiver Integration:** Replacing the wired speaker connection with wireless transceivers—such as an **ESP32 microcontroller** with integrated Bluetooth or a Wi-Fi module—would allow the handglove to transmit translation data directly to a smartphone app, smart speaker, or cloud database.
- **Implementation of Machine Learning (ML):** Currently, the system uses static conditional boundaries (e.g., analog value < 630). Implementing an onboard Machine Learning classifier (like a lightweight neural network or k-nearest neighbors algorithm on a Raspberry Pi or advanced microcontroller) would allow the system to recognize continuous, moving gestures rather than just static shapes.
- **Printed Circuit Board (PCB) Fabrication:** Migrating the circuit from a temporary prototype breadboard to a custom-designed, multi-layer printed circuit board (PCB) would drastically reduce the overall footprint, eliminate loose wiring connections, and make the glove highly durable.
- **Dual-Glove Synchronization:** Many sign language words require the coordinated movements of both hands. Linking a secondary left-hand glove to sync with the right-hand glove via an I^2C bus or wireless link would enable a fully comprehensive sign language dictionary.

BIBLIOGRAPHY

The following reference materials, textbooks, datasheets, and online documentation were consulted during the research, design, and construction of this investigative physics project:

1. Textbooks and Academic Literature

- **A Complete Course in ISC Physics (Class XII)** – *V.P Bhatnagar, Ahok Kumar Chawla and Priyanka Bhardwaj* (Pitambar)
    - Refered to for foundational concepts on: Ohm's Law, combinations of resistors in series and parallel, and electromagnetic induction.

2. Component Datasheets and Manuals

- **Spectra Symbol Flex Sensor Datasheet (2.2 Inch)** – *Spectra Symbol Corp.*
    - Consulted for nominal resistance ranges, power ratings, and bending physical tolerances.
- **MPU-6050 Product Specification & Register Map** – *InvenSense Inc.*
    - Consulted for I^2C registers, accelerometer sensitivities ($\pm$2g), and gyroscope configurations.
- **ATmega328P 8-bit Microcontroller Datasheet** – *Atmel/Microchip Technology*
    - Consulted for operating voltage limits, ADC mapping, and power dissipation parameters.

3. Online Resources and Development Ports

- **Arduino Official Documentation** – https://www.arduino.cc
    - Referenced for standard libraries, code syntax structures, and SoftwareSerial documentation.
- **DFRobot Wiki for DFPlayer Mini** – https://wiki.dfrobot.com
    - Referenced for the serial control command structures and wiring safety layouts.
- **Just Do Electronics (Sing - to speech - conversion)** - https://justdoelectronics.com/sign-language-to-speech-conversion#google_vignette
    - Referenced to understand the concept of sign language to speech conversion and how it's done.